



**Jose Said
Olano Garcia**

*Senior Software
Development Manager*

About the presenter



josesaid@gmail.com



linkedin.com/in/josesaid



Spring · Java · Microservices · AWS

About this presentation

A complete exploration of Spring AI 1.0 GA: fundamentals, ChatClient, RAG, Agents, Advisors, Memory, ETL Pipeline, Ollama, Observability, MCP and Testing. Real Java code in every slide.



Spring AI

*Generative AI for
the Spring Ecosystem*

Spring AI 1.0 GA

Contents

- What is Spring AI?
- ChatClient & Chat Models
- Structured Output
- Models & Providers
- Advisors
- ETL Pipeline
- Observability
- AI Testing
- Best Practices
- Architecture
- Prompt Templates
- RAG & Vector Stores
- AI Agents & Tool Calling
- Chat Memory
- Local Models (Ollama)
- MCP Protocol
- Multimodality
- Conclusions



What is Spring AI?

Purpose



Framework that simplifies building Generative AI applications in the Spring/Java ecosystem.

Inspiration



Inspired by LangChain and LlamaIndex (Python), but designed natively for Java enterprise.

Portability



Abstracts AI providers (OpenAI, Azure, Google...) with common, interchangeable interfaces.

Integration



Works natively with Spring Boot: auto-configuration, `application.properties` and starters.



Spring AI Architecture

Your Spring Boot Application



Spring AI Core · ChatClient · PromptTemplate · VectorStore · Advisors



AI Providers · OpenAI · Azure · Vertex · Bedrock · Ollama...



ChatClient



Prompt
Template



Vector
Store



Advisors



Output
Parser



AI Client & Chat Model

ChatClient

Fluent, portable API to interact with any LLM:

- Synchronous and streaming responses
- System, user and assistant messages
- Multi-turn conversation
- Advisors for pre/post-processing
- Multi-modal support (text + image)
- @Autowired ready with Spring Boot



```
@Autowired ChatClient chatClient;
```

```
// Simple response
```

```
String r = chatClient
```

```
    .prompt()
```

```
    .user("What is Spring AI?")
```

```
    .call().content();
```

```
// Streaming
```

```
chatClient.prompt()
```

```
    .user("Explain step by step")
```

```
    .stream()
```

```
    .content()
```

```
    .subscribe(out::println);
```



Prompt Templates



PromptTemplate

Reusable prompts with {placeholder} variables. Separates prompt text from business logic.



System + User

Combines system messages (role/behavior) with user messages to control context.



.st Files

Templates loaded from classpath as StringTemplate files — no recompilation needed.

```
PromptTemplate pt = new PromptTemplate(
    "Translate {text} to {language}");
Prompt p = pt.create(Map.of("text", "Hello", "language", "fr"));

// System + User
chatClient.prompt().system("You are a Java expert")
    .user("Explain records").call();
```



Structured Output

Spring AI automatically converts LLM responses into typed Java objects — no manual parsing



```
// Define your Java Record
record Actor(String name, List<String> movies) {}

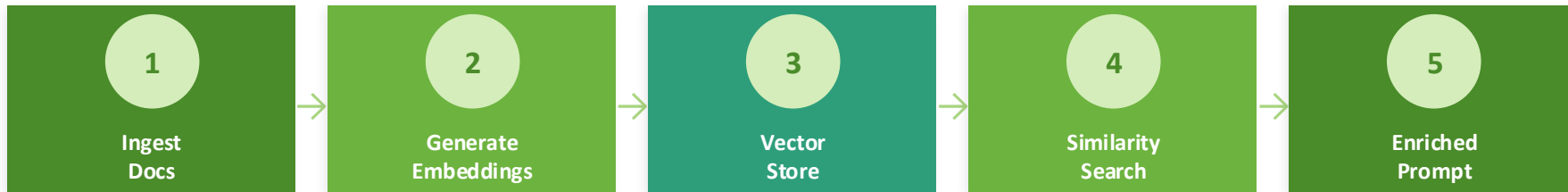
// Spring AI parses automatically
Actor actor = chatClient
    .prompt("Info about Keanu Reeves")
    .call()
    .entity(Actor.class); // no manual Jackson!

// Also: List<T> · Map<K,V> · BeanOutputConverter
```



RAG & Vector Stores

Retrieval Augmented Generation — Enrich responses with your own data



Supported Vector Stores:

PgVector

Redis

Chroma

Pinecone

Weaviate

Milvus

MongoDB Atlas

Azure AI Search

```
vectorStore.similaritySearch(SearchRequest.query("Java microservices").withTopK(5));
```




Models & Providers

Chat Models



GPT-4o, Claude 3.5, Gemini Pro, Llama 3, Mistral

Embedding Models



text-embedding-3, Titan Embeddings, Nomic, MiniLM

Image Generation



DALL-E 3, Stable Diffusion, Vertex Imagen

Speech & Audio



OpenAI Whisper, TTS — transcription and synthesis

All models share the same interfaces — switching providers requires no code changes



AI Agents & Tool Calling

Tool Calling

Allows the LLM to call real Java functions at runtime:

- Annotate methods with `@Tool` to expose them
- The model decides when and how to use them
- Static and dynamic tools
- Agents that use APIs, DBs and services
- Support for chained tool calls



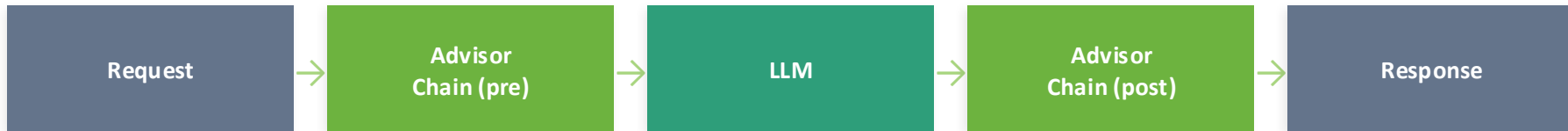
```
// Define a tool
@Tool(description =
    "Current stock price")
public double getPrice(String ticker) {
    return stockSvc.getPrice(ticker);
}

// Register in ChatClient
chatClient.prompt()
    .user("Analyze AAPL vs MSFT")
    .tools(new StockTools())
    .call().content();
```



Advisors & Interceptors

Intercept and modify conversation flow before and after the LLM



MessageChatMemoryAdvisor

Automatically injects conversation history into each prompt — transparent memory.



QuestionAnswerAdvisor

Auto RAG: queries the VectorStore and enriches the prompt context.



Custom Advisor

Implement CallAroundAdvisor for logging, security, moderation or caching.

```
chatClient.prompt()  
    .advisors(new MessageChatMemoryAdvisor(mem), new QuestionAnswerAdvisor(vs))  
    .user(query).call().content();
```



Chat Memory



InMemoryChatMemory

Stores history in RAM. Ideal for development and PoC. Cleared on restart.



JdbcChatMemory

Persists conversation in a relational DB via Spring JDBC. Production-ready.



CassandraChatMemory

Distributed storage on Apache Cassandra for high availability.



```
// Configure persistent memory
ChatMemory memory = new JdbcChatMemory(jdbcTemplate);

// Advisor keeps history per conversationId
chatClient.prompt()
    .advisors(new MessageChatMemoryAdvisor(memory))
```

History is automatically injected into every request — the LLM keeps context without extra code



Document ETL Pipeline

Three-phase pipeline to ingest documents into the Vector Store

1. Document Reader

Reads PDFs, HTML, Markdown, Word, JSON, CSV from disk, URL or classpath.

2. Document Transformer

Splits, cleans and enriches. `TokenTextSplitter`, `KeywordMetadataEnricher`...

3. Document Writer

Writes vectorized chunks to the Vector Store of your choice.

```
// Reader → Transformer → Writer
new TokenTextSplitter()
    .apply(new PagePdfDocumentReader("docs.pdf"))
    .read()
    .forEach(vectorStore::add);

// DocumentReaders: PDF, HTML, Markdown, TIKa, GitHub, YouTube...
```



Local Models with Ollama

Run LLMs locally — no external APIs, no token costs, no data leaving your infrastructure



Full Privacy

Data never leaves your infrastructure. Ideal for sensitive data or regulated environments.



Zero Token Cost

No per-token billing. Perfect for development, testing and batch processing.



Same API

Java code is identical. Just change the starter and properties — zero refactoring.



```
// application.properties
```

```
spring.ai.ollama.base-url=http://localhost:11434
```

```
spring.ai.ollama.chat.options.model=llama3
```

```
// ChatClient works exactly like with OpenAI
```

```
String r = chatClient.prompt().user("Hello!").call().content();
```

Models: Llama 3, Mistral, Phi-3, Gemma 2, CodeLlama, Qwen, DeepSeek...



Micrometer Metrics

Latency, token usage and errors exported to Prometheus, DataDog, CloudWatch...



Distributed Tracing

Traces with Micrometer Tracing (Zipkin, Jaeger) for the full request lifecycle.



Spring Actuator

/actuator/health and /actuator/metrics to monitor model health in production.



```
// application.properties
management.endpoints.web.exposure.include=health,metrics,info
management.tracing.sampling.probability=1.0

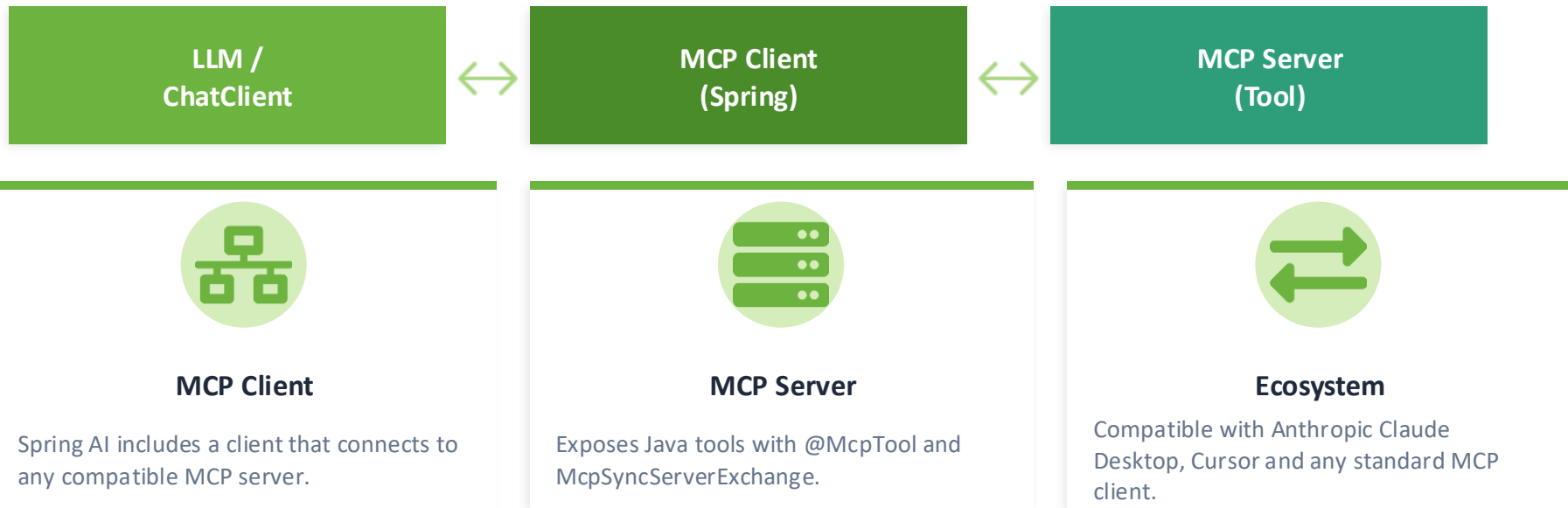
// Auto metrics available:
// spring.ai.chat.client.requests    (call counter)
```

Enabled by adding spring-boot-starter-actuator + micrometer-registry-prometheus



Model Context Protocol (MCP)

Standard protocol to connect LLMs with external tools and data sources



```
spring.ai.mcp.client.sse.connections.myServer.url=http://localhost:8080/mcp
```




AI Testing & Evaluation

Spring AI includes utilities to evaluate the quality and relevance of model responses



RelevancyEvaluator

Evaluates whether the response is relevant to the question using the LLM as judge.



FactCheckingEvaluator

Verifies the response is grounded in retrieved context (anti-hallucination).



MockChatClient

Simulates LLM responses in unit tests without real API calls.



```
@SpringBootTest
```

```
class AiTest {
```

```
    @Autowired RelevancyEvaluator evaluator;
```

```
    @Test void testRelevancy() {
```

```
        var req = new EvaluationRequest("Capital of Mexico?",
```

```
            List.of(), "Mexico City");
```

Multimodality

Send text, images, audio and documents to the model — all from the same ChatClient



Vision (Image)

Analyze images, screenshots and diagrams.
Compatible with GPT-4o Vision, Claude 3.5, Gemini.



Audio & Voice

Transcription (Speech-to-Text) and
synthesis (TTS) with OpenAI Whisper and
Azure Speech.



Documents

Attach PDFs and files as direct context in
the prompt for inline analysis.



```
// Analyze an image with GPT-4o Vision
var media = new Media(MimeTypeUtils.IMAGE_PNG,
    new ClassPathResource("diagram.png"));
String result = chatClient.prompt()
    .user(u -> u.text("What does this diagram show?"))
    .media(media))
```



Best Practices



Externalize prompts

Store templates in .st classpath files. Iterate without recompiling.



Enable Observability

Configure Micrometer from day one. Monitor latency and token usage in every env.



Use ChatMemory

JdbcChatMemory in production. InMemory only for dev and tests.



Ollama in development

No costs or network latency. Switch to cloud in production with zero refactor.



Integrate AI Testing

Add RelevancyEvaluator to CI/CD to catch response quality regressions.



Design for portability

Inject ChatClient via Spring. Never couple code to a specific provider SDK.



Conclusions

Spring AI 1.0 GA

docs.spring.io/spring-ai

github.com/spring-projects/spring-ai



Portability over 15+ AI providers



Native Spring Boot integration



Chat, RAG, Agents, Memory, Multimodal



Local models with Ollama — zero refactor



Observability with Micrometer from day one



Evaluators for anti-hallucination testing



MCP: integrate any external tool



From PoC to production without rewriting code